



**UNIVERSIDAD  
SERGIO ARBOLEDA**

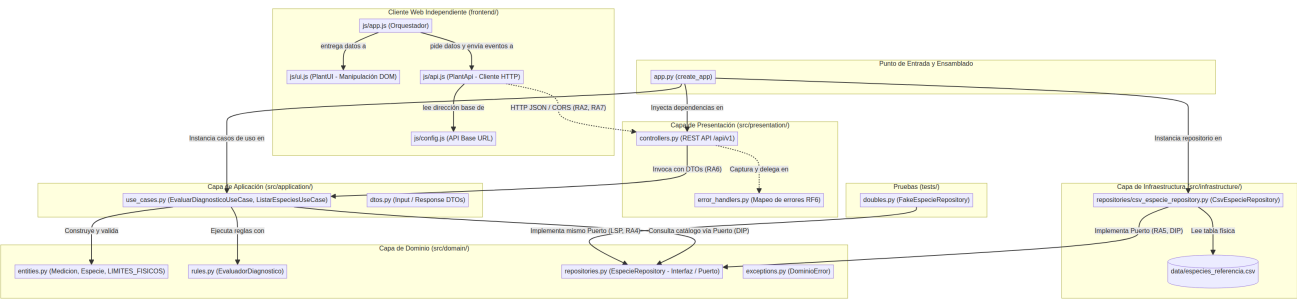
**Documento de Arquitectura de Software**  
**Sistema de Diagnóstico del Estado de Plantas**  
Proyecto de Corte — Arquitectura de Software

**Yslen Natalia Moreno**  
**Esteban Castro Rojas**  
**Juan Sebastián Gonzalez**

*Universidad Sergio Arboleda*  
*Escuela de ciencias exactas e ingeniería*  
*Bogotá D.C., septiembre de 2026*

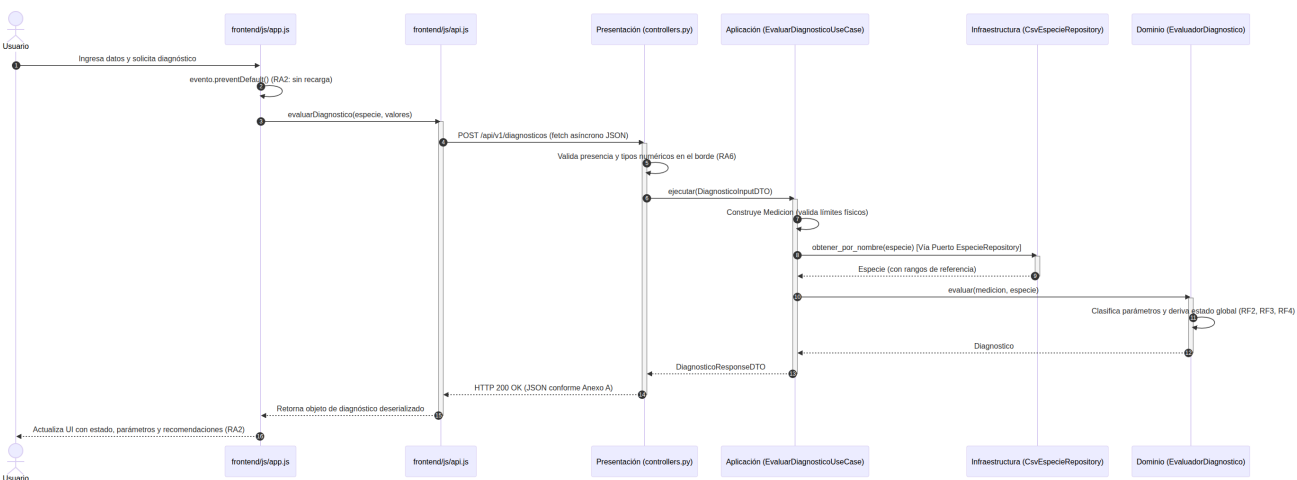
1. Diagrama de Paquetes / Componentes

El siguiente diagrama representa la estructura de componentes del repositorio, organizada en un cliente web independiente y cuatro capas en el backend con la dirección de dependencias orientada hacia el dominio:



2. Diagrama de Secuencia: Petición de Diagnóstico

Recorrido de ejecución completo para la operación de diagnóstico botánico (POST /api/v1/diagnosticos), nombrando los componentes y clases reales del repositorio:



3. Tabla de Responsabilidades por Capa

Cada capa tiene una única razón para cambiar y depende exclusivamente de capas más internas. Las prohibiciones de la tercera columna son verificables sobre el código mediante la inspección de los `import` de cada paquete.

| Capa                              | Qué hace                                                                                                                                                                                                                    | Qué tiene PROHIBIDO hacer                                                                                                                               | De qué depende                                                                    |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Presentación<br>src/presentation/ | Expone los endpoints REST (POST /api/v1/diagnosticos, GET /api/v1/especies), recibe peticiones HTTP, valida tipos básicos en el borde del sistema y mapea errores a respuestas JSON con el código de estado adecuado (RF6). | Renderizar HTML o plantillas de servidor (RA1), contener lógica de negocio o reglas de salud vegetal, acceder a archivos o bases de datos directamente. | Capa de <b>Aplicación</b> (casos de uso y DTOs) y excepciones de <b>Dominio</b> . |

| Capa                                          | Qué hace                                                                                                                                                                                                                                                                                                                                                 | Qué tiene PROHIBIDO hacer                                                                                                                                                                                | De qué depende                                                                                                                                            |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Aplicación</b><br>src/application/         | Orquesta los casos de uso (EvaluarDiagnosticoUseCase, ListarEspeciesUseCase), coordina la conversión entre DTOs y entidades de dominio, y consulta los datos mediante el puerto de repositorio.                                                                                                                                                          | Depender de frameworks web (Flask, Django), conocer detalles de persistencia (CSV, SQL), generar respuestas HTTP o conocer <i>requests</i> de red.                                                       | Capa de <b>Dominio</b> (entidades, excepciones, reglas y abstracciones de repositorio).                                                                   |
| <b>Dominio</b><br>src/domain/                 | Encapsula las entidades puras (Medicion, Especie), declara en <code>LIMITES_FISICOS</code> el catálogo de variables ambientales soportadas y sus invariantes físicas, deriva el estado global de la planta (RF3), clasifica variables (RF2), genera recomendaciones (RF4) y declara el contrato abstracto <code>EspecieRepository</code> ( <b>RA5</b> ). | Importar bibliotecas web (Flask, FastAPI), importar bibliotecas de persistencia (CSV, SQLite, Pandas), depender de capas externas ( <b>RA4</b> ). El dominio es 100 % puro y corre en cualquier entorno. | <b>De nada:</b> únicamente de la biblioteca estándar de Python ( <code>typing</code> , <code>dataclasses</code> , <code>enum</code> , <code>abc</code> ). |
| <b>Infraestructura</b><br>src/infrastructure/ | Implementa los adaptadores concretos de persistencia ( <code>CsvEspecieRepository</code> ) leyendo la tabla de referencia del CSV, gestiona la caché de acceso a datos y realiza las operaciones de E/S física.                                                                                                                                          | Decidir si una planta está sana o enferma, alterar reglas de clasificación de negocio, exponer controladores HTTP.                                                                                       | Capa de <b>Dominio</b> : implementa la interfaz abstracta <code>EspecieRepository</code> y produce entidades <code>Especie</code> .                       |
| <b>Frontend</b><br>frontend/                  | Cliente web desacoplado ( <code>config.js</code> , <code>api.js</code> , <code>ui.js</code> , <code>app.js</code> ): construye el formulario dinámicamente desde el catálogo (RF5), realiza peticiones asíncronas ( <code>fetch</code> ) sin recargar la página ( <b>RA2</b> ) y presenta el diagnóstico y los errores al usuario.                       | Conocer lógica de negocio agronómica o validar rangos biológicos en el cliente: la autoridad es del backend.                                                                                             | De la URL del backend ( <code>config.js</code> ) y del contrato JSON expuesto por la capa de Presentación.                                                |

### 3.1. Regla de agregación del estado global (RF3)

El requisito RF3 La regla implementada en `src/domain/rules.py` (`_derivar_estado_global`, Líneas 101–114) es un conteo de parámetros fuera de rango:

| Parámetros fuera de su rango óptimo | Estado global |
|-------------------------------------|---------------|
| Ninguno                             | SALUDABLE     |
| Uno                                 | EN_RIESGO     |
| Dos o más                           | CRITICO       |

**Justificación.** Un solo parámetro desviado suele ser corregible de forma aislada (regar, cambiar de ubicación), mientras que dos o más desviaciones simultáneas indican condiciones ambientales inadecuadas de fondo. La regla **no compensa**: no se promedian los parámetros ni se descuenta un factor adverso contra dos favorables, porque la tolerancia de una planta no funciona por promedios.

**Por qué no se emplean medidas estadísticas.** Se consideró derivar el estado de un promedio o de la desviación estándar de los valores medidos y se descartó: la desviación estándar describe la dispersión de una muestra de una misma variable, mientras que aquí existe una sola lectura de tres variables en unidades incomparables (% , lux, °C). Su resultado carecería de interpretación agronómica.

**Tensión reconocida.** La regla distingue únicamente si un parámetro está dentro o fuera del rango, no qué tan lejos: una humedad un punto por debajo del mínimo y un sustrato completamente seco producen el mismo estado. Se asumió por simplicidad y trazabilidad de la decisión. Incorporar la magnitud de la desviación implicaría modificar un único método privado del dominio, ya que ningún otro componente conoce la regla de agregación: los casos de uso, el controlador y el frontend reciben el estado ya calculado.

## 4. Justificación de Principios SOLID

El criterio que guio el diseño es que un cambio en el mecanismo de entrada (HTTP, MQTT) o en el mecanismo de almacenamiento (CSV, base de datos) no debe propagarse hacia las reglas de negocio. A continuación se justifica

cada principio con referencia a archivo y línea del repositorio.

## Single Responsibility Principle (SRP)

### Dónde se aplica:

- `src/domain/rules.py` (Líneas 19–114): la clase `EvaluarDagnostico` tiene una única razón para cambiar: cambios en las reglas agronómicas de evaluación o en la derivación del estado global de la planta.
- `src/infrastructure/repositories/csv_especie_repository.py` (Líneas 18–79): la clase `CsvEspecieRepository` solo cambia si varía el formato del archivo CSV o la estrategia de lectura del disco.
- `src/presentation/controllers.py` (Líneas 15–71): los controladores solo cambian si se modifican los contratos de transporte REST o la serialización HTTP.

Si una sola clase o script (`app.py` monolítico) leyera el CSV, calculara el estado y emitiera la respuesta HTTP, un cambio en el delimitador del archivo de texto rompería o exigiría volver a probar la lógica de diagnóstico botánico y el *endpoint* web.

## Open/Closed Principle (OCP)

### Dónde se aplica:

- `src/domain/entities.py` (Líneas 44–49): el dominio declara en `LIMITES_FISICOS` el catálogo de variables ambientales soportadas y deriva de él `PARAMETROS_SOPORTADOS` (L53).
- `src/domain/rules.py` (Líneas 67–100): `evaluar()` no enumera parámetros fijos; itera sobre `especie.rangos` (L77). Tanto `Especie.rangos` (L69–81) como `Medicion.valores` (L84–129) son colecciones indexadas por nombre (Mapping), no atributos estáticos acoplados.
- `src/infrastructure/repositories/csv_especie_repository.py` (Líneas 28, 35–37, 51–63): las columnas del CSV se derivan dinámicamente (`{parametro}_min` / `{parametro}_max`), requiriendo solo `_PREFIXOS_CSV` para la abreviación de temperatura del Anexo B.
- `src/presentation/controllers.py` (Líneas 49–52): el *endpoint* valida y exige los parámetros publicados dinámicamente por el dominio.
- `src/domain/repositories.py` (Líneas 10–24): la abstracción `EspecieRepository` permite incorporar nuevos orígenes de datos (PostgreSQL, dobles de prueba) sin modificar la capa consumidora.

**Verificación y extensibilidad:** agregar un nuevo parámetro ambiental (por ejemplo, pH del sustrato) requiere intervenir únicamente dos archivos: añadir la clave en `LIMITES_FISICOS` y agregar las columnas correspondientes en `data/especies_referencia.csv`. Ni el evaluador de reglas, ni los DTOs, ni los casos de uso, ni los controladores requieren modificación.

**Tensión de diseño reconocida:** el cálculo y el flujo de evaluación quedan cerrados a modificación, pero el catálogo de textos específicos en `EvaluarDagnostico._RECOMENDACIONES` (L31–44) requeriría edición si se desea una redacción personalizada para el nuevo parámetro; de lo contrario, aplica la recomendación genérica de `_generar_recomendacion()` (L59–64). Se asumió conscientemente para no acoplar el dominio a persistencia externa de cadenas.

**Qué habría pasado de no aplicarlo:** si `Especie` y `Medicion` usaran atributos fijos (`rango_humedad`, `rango_luz`, etc.), añadir una variable obligaría a modificar entidades, repositorios, evaluadores, casos de uso, DTOs y controladores, violando el cierre ante cambios.

## Liskov Substitution Principle (LSP)

### Dónde se aplica:

- `src/domain/repositories.py` (Líneas 10–24) frente a `src/infrastructure/repositories/csv_especie_repository.py` (Líneas 18–79) y `tests/doubles.py` (Líneas 29–46): tanto `CsvEspecieRepository` como `FakeEspecieRepository` implementan `EspecieRepository`.
- El caso de uso `EvaluarDagnosticoUseCase` (`src/application/use_cases.py`, Líneas 20–57; constructor L27–28) recibe `EspecieRepository` y funciona idénticamente con la implementación física o con el doble en memoria, cumpliendo los mismos contratos de entrada y salida sin lanzar excepciones inesperadas que rompan el cliente.

**Qué habría pasado de no aplicarlo:** si la implementación concreta de CSV lanzara excepciones específicas de archivo que la capa de aplicación tuviera que capturar con bloques `try/except csv.Error`, no se podría sustituir por un repositorio en base de datos o por un doble de pruebas sin modificar la capa de aplicación.

## Interface Segregation Principle (ISP)

### Dónde se aplica:

- `src/domain/repositories.py` (Líneas 10–24): la interfaz `EspecieRepository` es pequeña y especializada. Contiene única y exclusivamente los dos métodos que el dominio requiere: `obtener_por_nombre(nombre)` y `obtener_todas()`.

**Tensión / decisión consciente:** no se forzó una interfaz genérica de repositorio CRUD con métodos que el dominio actual no necesita (`guardar()`, `eliminar()`, `actualizar()`, `paginar()`). Las clases implementadoras solo escriben código relevante para la lectura de especies.

**Qué habría pasado de no aplicarlo:** si se hubiera utilizado una interfaz de repositorio estándar con diez métodos no utilizados, `CsvEspecieRepository` y los dobles de prueba estarían obligados a implementar métodos vacíos o a lanzar `NotImplementedError`, ensuciando el diseño.

## Dependency Inversion Principle (DIP)

### Dónde se aplica:

- Es la formalización de **RA5**. El caso de uso de aplicación (`src/application/use_cases.py`, Líneas 27–28) depende de la abstracción `EspecieRepository` definida en `src/domain/repositories.py` (Líneas 10–24).
- La implementación física `CsvEspecieRepository` vive en la capa de infraestructura (`src/infrastructure/repositories/csv_especie_repository.py`, Línea 18) y depende de la interfaz de dominio.
- La composición final se realiza en `app.py` (Líneas 36–46), donde se inyecta la instancia concreta en el caso de uso y en el *blueprint*.

**Qué habría pasado de no aplicarlo:** el caso de uso importaría directamente `CsvEspecieRepository`. Esto haría imposible ejecutar pruebas unitarias del caso de uso sin tener el archivo físico en el disco duro y violaría la restricción **RA4**.

## 5. Tabla de Referencia por Especie: Contenido y Fuente

La tabla de referencia (`data/especies_referencia.csv`) cubre cinco especies, el mínimo exigido, con el formato del Anexo B del enunciado:

| Especie     | Humedad (%) | Luz (lux)   | Temperatura (°C) |
|-------------|-------------|-------------|------------------|
| sansevieria | 20 – 45     | 200 – 1500  | 15 – 29          |
| potos       | 40 – 70     | 300 – 1200  | 18 – 30          |
| suculenta   | 10 – 30     | 800 – 2500  | 15 – 32          |
| helecho     | 60 – 85     | 150 – 800   | 16 – 26          |
| lavanda     | 25 – 50     | 1000 – 3000 | 15 – 30          |

**Fuente:** los valores fueron tomados íntegramente del **Anexo B del enunciado del proyecto de corte** (*Formato de la tabla de referencia*), sin modificación. No se realizó una verificación agronómica independiente de los rangos: el objeto de este corte es la separación entre la tabla y la lógica que la consume, no la exactitud botánica del dato. Esa decisión es en sí misma una demostración del diseño: corregir cualquiera de estos rangos contra una fuente especializada es editar un archivo CSV y no requiere tocar una sola línea de código.

Las unidades no viven en el CSV: las declara el dominio en `LIMITES_FISICOS` (`src/domain/entities.py`, Líneas 44–49) y la infraestructura las adjunta al construir cada `RangoParametro`.

## 6. Plan de Evolución

### Escenario A: las mediciones llegan por MQTT desde un ESP32

#### Qué se agrega:

- Un nuevo adaptador de entrada en la capa de presentación: `src/presentation/mqtt_subscriber.py`, utilizando una biblioteca cliente MQTT (p.ej. `paho-mqtt`).
- Un traductor que convierta el *payload* JSON recibido en el tópico MQTT a `DiagnosticoInputDT0`.
- Un script de ejecución o servicio *daemon* para el *listener* MQTT (p.ej. `mqtt_runner.py`).

**Qué se modifica:** ninguno de los componentes de negocio. Opcionalmente `app.py`, para registrar el cliente MQTT en segundo plano o gestionar variables de entorno de red.

**Qué NO se toca:** la totalidad de la capa de dominio (`entities.py`, `rules.py`, `exceptions.py`); la totalidad de la capa de aplicación (`use_cases.py`: `EvaluarDiagnosticoUseCase` es agnóstico al protocolo de transporte y se reutiliza idénticamente); la capa de infraestructura CSV.

## Escenario B: la tabla de referencia migra de CSV a PostgreSQL

**Qué se agrega:**

- Una nueva clase en la capa de infraestructura: `src/infrastructure/repositories/postgres_especie_repository.py`, que implementa `EspecieRepository` mediante SQL o un conector ligero (`psycopg2`, `SQLAlchemy core`).
- Script SQL de migración y creación de tabla con los datos semilla de las especies.

**Qué se modifica:** exclusivamente una línea en `app.py`: en lugar de instanciar `CsvEspecieRepository`, se instancia `PostgresEspecieRepository(connection_string)`.

**Qué NO se toca:** la interfaz `EspecieRepository` declarada en el dominio; los casos de uso; todas las entidades y reglas del dominio; todos los controladores REST y el frontend.

## Escenario C: aparecen usuarios, cada uno con varias plantas

**Qué se agrega:**

- Nuevas entidades en el dominio: `Usuario` y `PlantaUsuario` (identificador, apodo, especie asociada, `usuario_id`).
- Nuevas abstracciones de puerto en el dominio: `UsuarioRepository` y `PlantaUsuarioRepository`.
- Nuevos casos de uso: `RegistrarPlantaUseCase`, `ListarPlantasUsuarioUseCase`.
- Nuevas tablas e implementaciones en infraestructura para la persistencia de usuarios y plantas.
- Nuevos *endpoints* REST: `/api/v1/usuarios` y `/api/v1/plantas`.

**Qué se modifica:** el caso de uso `EvaluarDiagnosticoUseCase` puede, opcionalmente, recibir el `planta_id` para resolver la especie asociada en lugar de recibirla directamente.

**Qué NO se toca:** `EvaluarDiagnostico` —la regla para determinar si una humedad o una temperatura es óptima no cambia en función de quién sea el dueño de la planta—, `Medicion` y los límites físicos de las variables, ni la definición de especie y sus rangos de referencia.

## Escenario D: se agrega gamificación (puntos, rachas, niveles)

**Qué se agrega:**

- Un paquete de gamificación independiente: `src/gamification/`, con entidades `Puntaje`, `RachaCuidado` y `Nivel`, y reglas de otorgamiento de insignias.
- Un mecanismo de eventos de dominio en la capa de aplicación: `DiagnosticoGeneradoEvent`.
- Un suscriptor `GamificacionListener`, que incrementa la racha cuando el estado global es `SALUDABLE` o la penaliza si es `CRITICO`.

**Qué se modifica:** el caso de uso `EvaluarDiagnosticoUseCase`, para emitir el evento tras completar el diagnóstico.

**Qué NO se toca:** `src/domain/rules.py` y `src/domain/entities.py`. La botánica no conoce qué es un «nivel», una «racha» o una «medalla». Mezclar la gamificación dentro del evaluador de plantas violaría el SRP y contaminaría el núcleo biológico.

## 7. Decisiones de Diseño y Alternativas Descartadas

### Decisión 1: abstracción de repositorio en el dominio frente a carga directa de datos en los casos de uso

- **Opción adoptada:** declarar la interfaz abstracta `EspecieRepository` en `src/domain/repositories.py` e implementarla en `src/infrastructure/repositories/csv_especie_repository.py` (DIP / RA5).
- **Alternativa descartada:** leer directamente el CSV en el constructor del caso de uso o mediante funciones globales en un módulo utilitario.
- **Razón del descarte:** la alternativa acoplaba la lógica del sistema al sistema de archivos y al formato CSV. Habría hecho imposible probar los casos de uso sin crear archivos temporales en disco y obligaría a modificar la capa de aplicación al

migrar a una base de datos relacional, violando RA4 y RA5.

## Decisión 2: cliente web estático desacoplado frente a renderizado de plantillas con Jinja

- **Opción adoptada:** cliente web estático e independiente en `frontend/` (HTML, CSS y JavaScript puro) que consume la API de Flask mediante `fetch` asíncrono con CORS habilitado (RA1, RA2, RA7).
- **Alternativa descartada:** renderizado del formulario y de la tabla de resultados con `render_template` de Jinja2, servido directamente por las rutas de Flask.
- **Razón del descarte:** el renderizado del lado del servidor genera un acoplamiento monolítico entre la interfaz gráfica y el backend. Además de estar explícitamente prohibido bajo causal de descalificación por **RA1**, impedía que clientes futuros (aplicación móvil, paneles IoT) reutilizaran el servicio sin recibir HTML mezclado con los datos.

## Decisión 3: validación de invariantes físicos en el dominio frente a validación en el frontend o el controlador

- **Opción adoptada:** validación en dos barreras. El controlador REST valida obligatoriedad y formato de tipos (RA6); la entidad de dominio `Medicion` valida los invariantes y límites físicos de medición (RF6).
- **Alternativa descartada:** validar únicamente en el formulario HTML con JavaScript, o validar los rangos físicos con condiciones sueltas dentro de la ruta de Flask.
- **Razón del descarte:** si la validación reside solo en el front o en el controlador HTTP, al ingresar datos por MQTT desde el ESP32 o desde una prueba unitaria el sistema podría procesar datos absurdos (500 % de humedad, lux negativos). El dominio debe proteger su propia consistencia con independencia de la vía de entrada.

## Decisión 4: parámetros como colección indexada frente a campos fijos en las entidades

- **Opción adoptada:** `Especie.rangos` y `Medicion.valores` son colecciones indexadas por nombre de parámetro (`Mapping[str, ...]`), y el dominio publica el catálogo de variables soportadas en `LIMITES_FISICOS` (`src/domain/entities.py`, Líneas 44–49). El evaluador recorre lo que la especie declara.
- **Alternativa descartada:** mantener tres atributos fijos por entidad (`humedad`, `luz`, `temperatura`), que era el diseño original del repositorio.
- **Razón del descarte:** con campos fijos, agregar una cuarta variable obligaba a modificar siete archivos, entre ellos `EvaluadorDiagnostico`, lo que contradecía nuestra propia afirmación de OCP. Con la colección son dos, y ninguno es el evaluador.
- **Costo asumido:** se pierde el acceso por atributo (`medicion.humedad` pasa a `medicion.valor_de("humedad")`) y con él la detección de errores de tipeo en tiempo de escritura. Se compensa haciendo que `Medicion` valide en su constructor que cada clave exista en `LIMITES_FISICOS` (L104–108): un nombre inválido falla al construir la entidad, señalando el campo, y no silenciosamente durante la evaluación.
- **Nota sobre RA6:** `Medicion.valores` no es un diccionario crudo. `Medicion` es un objeto de valor que solo puede existir si supera sus invariantes (parámetro reconocido, valor numérico, valor físicamente posible) y congela su contenido con `MappingProxyType` (L122). El diccionario es su representación interna; su contrato de entrada es el constructor que valida.

## Decisión 5: front separado en capas frente a un único archivo de script

- **Opción adoptada:** dividir el cliente en cuatro archivos con una responsabilidad cada uno: `config.js` (dirección del backend), `api.js` (`PlantApi`, único que usa `fetch` y lee códigos HTTP), `ui.js` (`PlantUI`, único que toca el DOM) y `app.js` (orquestración). Los campos de medición del formulario se generan a partir del catálogo de RF5.
- **Alternativa descartada:** mantener `script.js`, un único archivo que mezclaba la configuración, las peticiones, el renderizado y el manejo de errores, con los tres campos escritos a mano en `index.html`.
- **Razón del descarte:** la separación del backend perdía sentido si el cliente concentraba todo en un archivo. Con la división, `ui.js` no sabe que existe HTTP —recibe datos ya resueltos y devuelve lo que el usuario escribió— del mismo modo que el dominio no sabe que existe Flask. Además, con los campos escritos a mano, agregar una variable ambiental obligaba a editar el HTML y el JS; al generarlos desde RF5, el front la incorpora sin cambios.
- **Costo asumido:** el front hace cuatro peticiones de script en lugar de una, y las etiquetas de los campos se derivan del nombre y la unidad que envía la API (`Humedad (%)`) en vez de textos redactados a mano. Para un cliente de este tamaño, servido localmente, ninguno de los dos costos es significativo.